TikuKits ML: Linear SVM for Embedded Systems

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

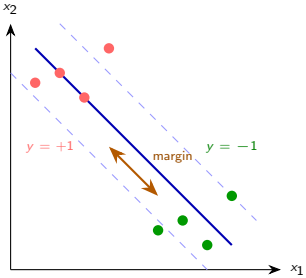
March 14, 2026

Outline

Why Linear SVM on MCUs?

Embedded Use Cases

- **Maximum-margin classification** — robust to noise
- **Binary decisions** — threshold/fault detection
- **L2 regularization** — prevents overfitting on small data
- **Margin inspection** — confidence from signed distance



Constraints

- No FPU (16-bit MSP430)
- No heap allocation
- Limited RAM (2–8 KB)

Design Goal

Binary linear SVM with **Pegasos-style online SGD**, hinge loss, and L2 regularization. Labels are $\{+1, -1\}$.

Linear SVM with Hinge Loss

Decision function:

$$f(\mathbf{x}) = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

Prediction:

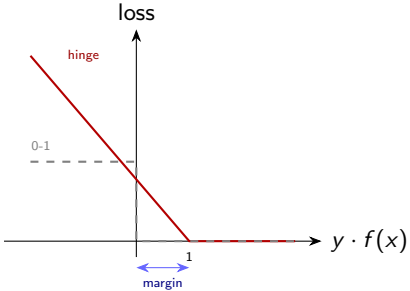
$$\hat{y} = \begin{cases} +1 & \text{if } f(\mathbf{x}) \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

Hinge loss + L2 regularization:

$$L = \max(0, 1 - y \cdot f(\mathbf{x})) + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

SGD update (Pegasos):

- If $y \cdot f(\mathbf{x}) < 1$ (margin violation):
  $w_j \leftarrow w_j + \eta(y \cdot x_j - \lambda \cdot w_j)$



Key Properties

- Hinge loss is convex and subdifferentiable
- L2 regularization prevents overfitting
- λ controls margin width vs. accuracy

The tiku_kits_ml_linsvm Structure

ml/classification/tiku_kits_ml_linsvm.h

```c
struct tiku_kits_ml_linsvm {
    /* bias + feature weights Q(shift) */
    int32_t weights[
        TIKU_KITS_ML_LINSVM_MAX_FEATURES
        + 1];
    uint8_t  n_features;
    uint8_t  shift;
    int32_t  learning_rate;
    int32_t  lambda;
    uint16_t n;
};
```

Memory footprint:
$(8+1) \times 4 + 1 + 1 + 4 + 4 + 2 = 48$ bytes.
All statically allocated — no heap.

Defaults at init (Q8)

```c
/* Learning rate: ~0.1 */
lr = (1 << shift) / 10;    /* = 25 */

/* Regularization: ~0.01 */
lambda = (1 << shift) / 100; /* = 2 */
```

Labels: $\{+1, -1\}$

Unlike logistic regression ($\{0, 1\}$), SVM uses $y \in \{+1, -1\}$. The `train()` function validates this — rejects any other label value.

Decision Margin

`decision()` returns the **raw signed margin** $f(\mathbf{x})$. Large positive = confident $+1$; large negative = confident -1.

Complete API

Function	Description
`tiku_kits_ml_linsvm_init(svm, nf, sh)`	Initialize with `nf` features, Q-format `sh`
`tiku_kits_ml_linsvm_reset(svm)`	Zero all weights, reset training count
`tiku_kits_ml_linsvm_set_lr(svm, rate)`	Set learning rate in Q(shift)
`tiku_kits_ml_linsvm_set_lambda(svm, lam)`	Set L2 regularization strength
`tiku_kits_ml_linsvm_train(svm, x, y)`	One Pegasos SGD step; label $y \in \{+1, -1\}$
`tiku_kits_ml_linsvm_predict(svm, x, &result)`	Binary prediction: $+1$ or -1
`tiku_kits_ml_linsvm_decision(svm, x, &margin)`	Raw signed margin $f(\mathbf{x})$ in Q(shift)
`tiku_kits_ml_linsvm_get_weights(svm, w)`	Export weight vector (bias + features)
`tiku_kits_ml_linsvm_set_weights(svm, w)`	Load pre-trained weights
`tiku_kits_ml_linsvm_count(svm)`	Number of training samples seen

Labels Must Be ± 1

`train()` returns `ERR_PARAM` if y is not exactly $+1$ or -1. This differs from logistic regression which uses $\{0, 1\}$.

Pegasos SGD Training

tiku_kits_ml_linsvm.c — train (simplified)

```c
int tiku_kits_ml_linsvm_train(struct tiku_kits_ml_linsvm *svm,
                              const tiku_kits_ml_elem_t *x, int8_t y)
{
    int32_t margin;
    int64_t lr_q = (int64_t)svm->learning_rate;
    int32_t one_q = (int32_t)(1 << svm->shift);

    margin = dot_product(svm, x);   /* f(x) in Q(shift) */

    if ((int64_t)y * (int64_t)margin < (int64_t)one_q) {
        /* Margin violation: hinge loss gradient */
        for (i = 0; i < svm->n_features; i++) {
            svm->weights[i + 1] +=
                (int32_t)((lr_q * ((int64_t)y * (int64_t)x[i]
                          - ((int64_t)svm->lambda * (int64_t)svm->weights[i + 1])
                            >> svm->shift)) >> svm->shift);
        }
    } else {
        /* Correct side: regularization only */
        for (i = 0; i < svm->n_features; i++) {
            svm->weights[i + 1] -=
                (int32_t)((lr_q * (int64_t)svm->lambda
                          * (int64_t)svm->weights[i + 1])
                          >> (2 * svm->shift));
```

Example 1: Temperature Threshold

examples/example_ml_linsvm.c — hot/cold binary classification

```c
struct tiku_kits_ml_linsvm svm;
int8_t result;

tiku_kits_ml_linsvm_init(&svm, 1, 8);   /* 1 feature, Q8 */

/* Train: cold(-1) / hot(+1) */
static const int32_t temps[]  = {150, 180, 200, 300, 320, 350};
static const int8_t labels[]  = { -1,  -1,  -1,  +1,  +1,  +1};

uint8_t epoch, s;
for (epoch = 0; epoch < 20; epoch++) {
    for (s = 0; s < 6; s++) {
        tiku_kits_ml_linsvm_train(&svm, &temps[s], labels[s]);
    }
}

/* Predict */
int32_t reading = 280;
tiku_kits_ml_linsvm_predict(&svm, &reading, &result);
/* result: +1 (hot) or -1 (cold) */
```

Example 2: Motion Detection with Margin

2-D with decision margin

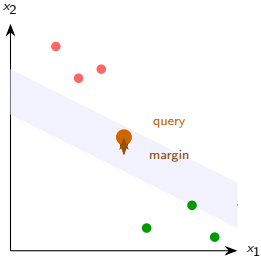
```c
struct tiku_kits_ml_linsvm svm;
int8_t result;
int32_t margin;

tiku_kits_ml_linsvm_init(&svm, 2, 8);

/* Train on accelerometer data */
/* ... training loop ... */

/* Classify and inspect margin */
tiku_kits_ml_elem_t sample[] = {15, 20};
tiku_kits_ml_linsvm_predict(&svm,
    sample, &result);
tiku_kits_ml_linsvm_decision(&svm,
    sample, &margin);

/* margin: signed distance in Q8 */
if (margin > 128) {
    /* Confident positive (> 0.5) */
    trigger_motion_alert();
} else if (margin < -128) {
    /* Confident negative */
    enter_sleep_mode();
} else {
```



Margin as Confidence

The absolute margin $|f(\mathbf{x})|$ indicates distance from the decision boundary. Large margin = high confidence. Use it for confidence-aware decisions.

Test Cases Overview

#	Test Function	What It Verifies
1	`test_kits_ml_linsvm_init`	Proper initialization, parameter validation
2	`test_kits_ml_linsvm_pretrained`	Pre-loaded weights produce correct ± 1
3	`test_kits_ml_linsvm_decision`	Raw decision margin sign and magnitude
4	`test_kits_ml_linsvm_training`	SGD convergence on linearly separable data
5	`test_kits_ml_linsvm_two_features`	2-D classification
6	`test_kits_ml_linsvm_learning_rate`	Custom learning rate accepted
7	`test_kits_ml_linsvm_lambda`	Custom regularization strength
8	`test_kits_ml_linsvm_reset`	Reset clears weights and count
9	`test_kits_ml_linsvm_null_inputs`	All functions reject NULL with ERR_NULL

Summary

What We Built

- Binary linear SVM classifier
- Pegasos-style online SGD
- Hinge loss + L2 regularization
- 48 bytes of state, no heap
- Raw margin output for confidence

Key Design Choices

- `int64_t` dot product prevents overflow
- Labels $\{+1, -1\}$ (validated)
- Separate `predict()` and `decision()`
- Tunable λ for regularization strength

Use Cases

- Temperature threshold classification
- Motion/vibration binary detection
- Margin-based confidence decisions
- Noise-robust sensor classification

Files

Header:	`ml/classification/tiku_kits_ml_linsvm.h`
Source:	`ml/classification/tiku_kits_ml_linsvm.c`
Tests:	`tests/ml/test_kits_ml_linsvm.c`
Example:	`examples/example_ml_linsvm.c`

Thank You

TikuKits ML: Linear SVM

Integer-Only Machine Learning for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
Common:	`tikukits/ml/tiku_kits_ml.h`
Header:	`tikukits/ml/classification/tiku_kits_ml_linsvm.h`
Source:	`tikukits/ml/classification/tiku_kits_ml_linsvm.c`
Tests:	`tikukits/tests/ml/test_kits_ml_linsvm.c`
Examples:	`tikukits/examples/example_ml_linsvm.c`